

# Immersive Gallery

Perceptual and Feature-Driven Navigation

Deliverable 2 — Final Report

Multimedia Computing

FCT, Universidade Nova de Lisboa

2025/2026

Tomas Fernandes Alexandre 73213

Nicolae Iachimovski 73381

May 29, 2026

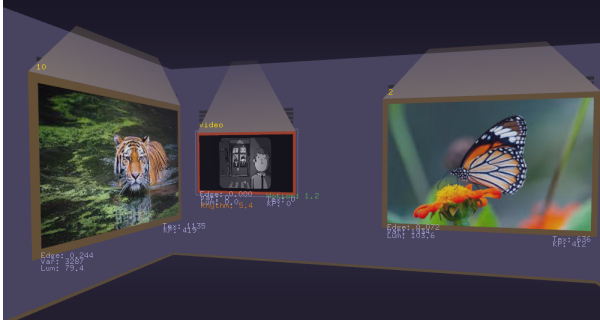
## Introduction

---

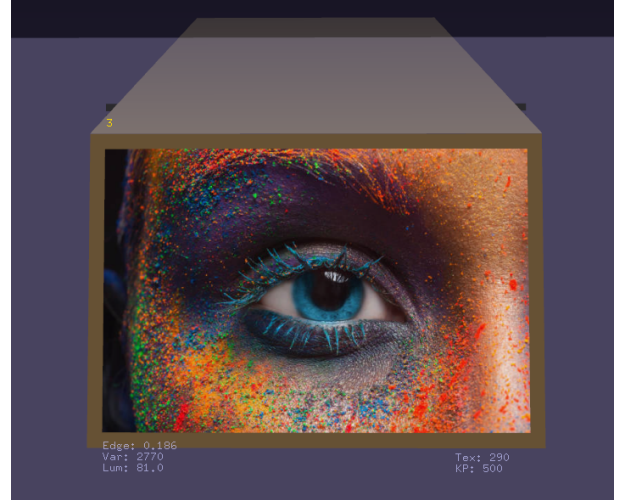
The Immersive Gallery is a wall-sized interactive installation that allows a user to navigate and explore a collection of images and videos displayed as virtual frames inside a 3D gallery space. The system goes beyond a static media viewer: it is a dynamic, feature-driven environment where visual and temporal properties extracted from each media item influence spatial organisation, grouping, interaction, and immersion.

The application is built in C++ using openFrameworks 0.12.1 and runs in two complementary modes. In **2D mode**, media items are displayed as a thumbnail grid with hover zoom, luminance and edge density bars, and fullscreen playback. In **3D mode**, the gallery becomes a navigable virtual room with paintings hung on walls, gradient spotlight beams, floating metadata labels, depth fog, and audio that varies with the camera’s distance to each frame. The user interacts via keyboard, mouse, and a live webcam feed that supports face detection and gesture recognition.

The system extracts seven perceptual features from each media item, namely mean luminance, luminance variance, edge density, number of ORB keypoints, texture (local standard deviation), motion energy, and video rhythm, stores them in individual XML files, and uses the same values to drive overlays, spatial organisation, similarity grouping, and at least four distinct spatial behaviours.



(a) 3D gallery view with paintings, spotlights, and fog.



(b) Floating metadata and feature bars visible on each frame.

Figure 1: The Immersive Gallery in 3D mode.

## Algorithms and Techniques

---

### Feature extraction

All feature extraction is handled by the `FeatureExtractor` class using OpenCV via the `ofxCv` addon. For each image, the pixel buffer is converted to a greyscale `cv::Mat` and the following descriptors are computed.

**Mean luminance** is the perceptual average pixel intensity: the image is converted to greyscale using `cv::COLOR_RGB2GRAY` (which applies the standard  $0.299R + 0.587G + 0.114B$  weighting) and

`cv::mean()` returns the scalar mean of the resulting channel. **Luminance variance** is the mean squared deviation from that mean, measuring global contrast. **Edge density** is computed using the Canny edge detector (`cv::Canny()`, thresholds 100/200) and is expressed as the ratio of edge pixels to total pixels. **Keypoints** are detected with OpenCV’s ORB detector (500-point limit), returning the count of interest points. **Texture** is estimated as the mean local variance: a blurred copy of the image is subtracted from the original, the residual is squared pixel-wise, blurred again with a  $9 \times 9$  kernel, and the scalar mean is taken.

For videos, **motion energy** is computed per frame as the mean absolute difference between the current and previous greyscale frames (`cv::absdiff`). **Video rhythm** is computed as the standard deviation of the motion energy values over a rolling window of 30 frames, capturing the variability of movement over time.

### Similarity and grouping

The **SimilarityEngine** class computes a weighted Euclidean distance between two **Features** structs. Each feature is normalised to  $[0, 1]$  before comparison. The distance function is:

$$d(a, b) = \sqrt{\sum_i w_i \left( \frac{f_i^a - f_i^b}{range_i} \right)^2}$$

where  $w_i$  are per-feature weights (edges weighted at 2.0, others at 0.5–1.0). Automatic grouping uses a greedy clustering algorithm: items are assigned to a group if their distance to the group seed is below a configurable threshold (default 0.3). In fullscreen view, the three most similar items are listed with their distances.

### Feature-driven spatial behaviours

Four distinct features control four distinct spatial behaviours. (1) **Mean luminance** controls the ordering of frames along the walls: items are sorted from darkest to lightest before being assigned wall positions. (2) **Luminance variance** controls the thickness of each frame’s border (8–24 px): high-contrast images receive a thicker border. (3) **Edge density** introduces a perceptual position offset of up to  $\pm 15$  units along the wall, so visually busier images are nudged slightly apart from their neighbours; the base layout distributes frames evenly within  $\pm 450$  units of each wall’s centre, with a computed `displayScale` factor ensuring all items fit regardless of collection size. (4) **Motion energy** controls the real-time scale of video frames: a high-motion frame expands by up to 50% of its base size. Additionally, **video rhythm** controls the colour of the video frame border, shifting from brown to red as rhythm intensity increases.

A fifth behaviour drives visual diversity in the 3D layout: the number of **ORB keypoints** and the **texture** score are combined into a complexity index that scales each frame’s rendered size between  $0.75\times$  and  $1.25\times$  its base dimensions:

$$sizeVariation = 0.75 + \frac{kpNorm + texNorm}{2} \times 0.5$$

Images with high visual complexity appear larger on the wall; simple or flat images appear smaller, producing a natural, museum-like variation in frame proportions. The wall-safety margin (`safeSpan`) always accounts for the maximum possible size ( $1.25\times$ ) to prevent frames from clipping into adjacent walls.

## Gesture detection

The `GestureDetector` class processes each webcam frame by computing a binary motion mask via `cv::absdiff` and thresholding. The centre of mass of the motion mask is tracked over a history of 10 frames. A horizontal displacement above 60 pixels is classified as a swipe left or right; a vertical displacement above 40 pixels upward is classified as a forward gesture; a motion energy above 25% of the frame is classified as strong motion.

## Presence and immersion

Depth fog is implemented using OpenGL’s fixed-function `GL_FOG` pipeline in exponential-squared mode. Its colour matches the wall colour (normalised RGB 0.282, 0.263, 0.373) so that distant objects fade into the background rather than turning black, preserving the room’s visual continuity at any viewing distance.

Spotlights are rendered as 2D gradient trapezoid meshes (`ofMesh`, `OF_PRIMITIVE_TRIANGLE_STRIP`) drawn in each frame’s local coordinate space. The beam widens from a narrow point ( $\approx 18$  px wide) above the frame down to exactly the frame’s full rendered width including its border, and fades from fully transparent at the top to warm amber (`rgba(255, 230, 150, 80)`) at the base, simulating a directed gallery spotlight without any visible solid geometry.

In 3D mode, hovering the mouse over a frame enlarges it by 30% via a `hoverMult = 1.3` scale factor applied to the frame’s rendered dimensions. Each frame’s world-space centre is projected to screen coordinates every frame using `cam.worldToScreen()`, and the closest frame within 140 px of the cursor is selected as the hover target. A yellow border glow is also drawn around the hovered frame for visual feedback.

Audio for video items is attenuated linearly with the camera’s distance to each frame (maximum volume at 200 units, silence at 1000 units). Floating metadata labels are rendered in the 3D coordinate space of each frame using `ofDrawBitmapString` before `ofPopMatrix`.

## Application Implementation

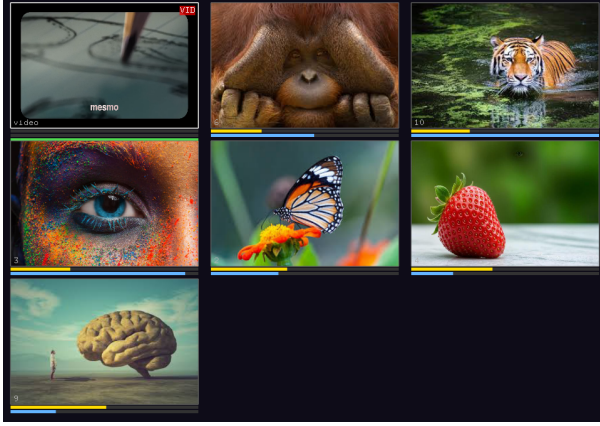
---

On startup, `ofApp::setup()` initialises the 3D camera at position (0,0,400) with a far clip of 5000 units, lists available webcam devices (selecting Camo if present), and loads all media from `bin/data/` (supporting .jpg, .jpeg, .png, .mp4, .mov, .avi). For each file, `loadXML()` checks whether a corresponding XML metadata file exists in `bin/data/metadata/`. If it does, the stored feature values are loaded directly; otherwise, `FeatureExtractor::extractFromImage()` is called and the results are saved. After loading, items are sorted by luminance and arranged on the four walls of the room.

`arrangeGallery()` distributes frames evenly across the four walls within  $\pm 450$  units of each wall’s centre. It computes a `displayScale` factor for each item proportional to the available slot size, so that any number of media files — even 100 or more — fit within the room without overlapping. It also computes a per-item `sizeVariation` multiplier (range 0.75–1.25 $\times$ ) based on ORB keypoint count and texture score, so that visually complex images appear larger than simple ones. The wall-clearance margin (`safeSpan`) is calculated using the maximum possible rendered size to guarantee that no frame clips into an adjacent wall regardless of its variation factor. Each item’s edge density

then offsets its position by up to  $\pm 15$  units within its slot. The same adaptive scaling is applied when similarity-based grouping is active via `groupAndArrange()`.

The `update()` loop advances video playback, computes real-time motion energy and rhythm for each video frame, adjusts video volumes based on camera distance, and processes webcam input for face detection and gesture recognition. The `draw()` loop renders either the fullscreen view, the 3D room (with optional fog, spotlights, frames, and metadata), or the 2D thumbnail grid depending on the current state.



(a) 2D grid mode with luminance (yellow) and edge density (blue) bars.



(b) Fullscreen view showing all features and similar items.

Figure 2: 2D mode and fullscreen view.

User interaction is handled via `keyPressed()`, `mousePressed()`, and `mouseMoved()`. In 3D mode, the arrow keys move the camera directly in front of the previous or next frame (wall-aware positioning via `getFrameCenter()`), and Enter opens the current frame in fullscreen. In 2D mode, clicking a thumbnail opens it in fullscreen and hovering enlarges it with a yellow border. The G key toggles similarity-based grouping, which rearranges frames by group on the walls and colours their borders by group index.

## Class Description

**Features (struct)** holds all extracted metadata for a single media item: `meanLuminance`, `luminanceVariance`, `edgeDensity`, `numKeypoints`, `texture`, `motionEnergy`, and `videoRhythm`, all as float or int values.

**MediaItem (struct)** represents a single image or video. It contains the file path, name, and type flag; an `ofImage` or `ofVideoPlayer`; a cached `ofTexture`; a 3D `ofVec3f` position; a `wallIndex` (0–3) identifying which wall the item is mounted on; a `displayScale` factor computed by `arrangeGallery()` to adapt frame size to collection size; a `sizeVariation` multiplier ( $0.75\text{--}1.25\times$ ) driven by visual complexity (keypoints and texture) to produce varied frame proportions on the wall; the **Features** struct; and video-specific state (`prevVideoFrame`, `hasPrevFrame`, `currentMotionEnergy`, `motionHistory`).

**ofApp** is the main application class. It owns the `vector<MediaItem>`, instances of `FeatureExtractor`,

**SimilarityEngine**, and **GestureDetector** as helper objects, the two camera objects (**ofEasyCam** for 3D navigation, **ofVideoGrabber** for the webcam), an **ofxCv::ObjectFinder** for face detection, and all UI state flags. It implements **setup()**, **update()**, **draw()**, **keyPressed()**, **mousePressed()**, and **mouseMoved()**, as well as **loadMedia()**, **saveXML()**, **loadXML()**, **sortByLuminance()**, **arrangeGallery()**, **groupAndArrange()**, **drawFrames()**, **drawFloor()**, **drawWalls()**, and the helper **getFrameCenter(int i)**, which returns the world-space centre of a frame offset slightly away from its wall surface, used for hover picking and camera navigation.

**FeatureExtractor** encapsulates all OpenCV-based feature computation. Its public interface exposes **extractFromImage(ofImage&)** returning a **Features** struct, and **extractMotionEnergy(ofPixels&, ofPixels&)** returning a float. Private methods handle each individual descriptor.

**SimilarityEngine** computes pairwise distances between **Features** structs using a weighted Euclidean metric. It exposes **distance()**, **findSimilar()** (returns the N nearest items), and **groupBySimilarity()** (greedy clustering). Feature weights are public fields, allowing runtime adjustment.

**GestureDetector** processes webcam pixel buffers to detect motion-based gestures. It maintains a frame-to-frame difference pipeline and a centre-of-mass history to classify **GESTURE\_SWIPE\_LEFT**, **GESTURE\_SWIPE\_RIGHT**, **GESTURE\_STRONG\_MOTION**, and **GESTURE\_FORWARD**. It also exposes raw motion energy for display.

The class diagram is included in Appendix A.

## Conclusions

---

The Immersive Gallery achieves the project’s core goal of creating a spatial, feature-driven media installation. The system successfully extracts seven perceptual descriptors from images and videos, stores them in XML, and uses the same values to drive four distinct spatial behaviours, similarity grouping, and visual overlays — satisfying the requirement that metadata values must be shared across all system functions.

The 3D gallery mode, combined with fog, gradient spotlight beams, audio attenuation, hover expansion, and floating metadata, produces a meaningful sense of immersion. The adaptive frame scaling ensures the installation remains usable with any number of media items, while the complexity-driven size variation gives the wall layout a natural, museum-like aesthetic where visually richer images occupy more space. The similarity engine provides coherent automatic grouping even with small media collections, and the real-time motion energy and rhythm features make video items visually responsive to their content.

The main limitation is the gesture interaction system, which depends on a functional webcam. The **GestureDetector** is fully implemented and tested, but hardware constraints prevented live camera testing on the development machine. In a standard setup with a working webcam, all gestures operate as designed. Future improvements could include edge histogram descriptors for richer similarity, smooth interpolated camera transitions between frames (rather than teleporting), and a fog density that varies automatically with the luminance of the nearest frame.

## References

---

1. Bradski, G., Kaehler, A. *Learning OpenCV 3*, O'Reilly Media, 2016.
2. Rublee, E. et al. “ORB: An efficient alternative to SIFT or SURF.” *ICCV*, 2011.
3. Kyle McDonald. *ofxCv*. <https://github.com/kylemcdonald/ofxCv>, 2023.
4. openFrameworks. *openFrameworks 0.12.1 documentation*. <https://openframeworks.cc/documentation/>, 2024.

## Appendix A — Class Diagram

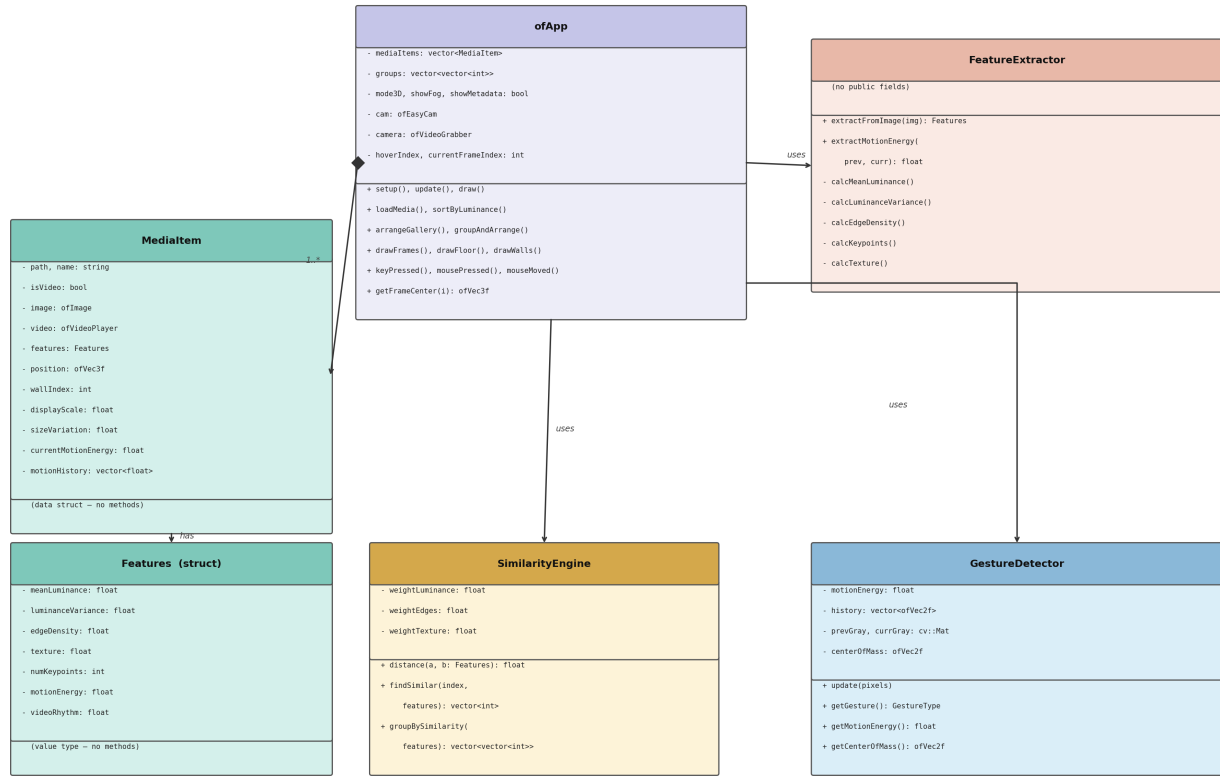


Figure 3: Class diagram of the Immersive Gallery system.

### Relationships:

- ofApp has a composition relationship with MediaItem (owns a vector of items).
- ofApp uses FeatureExtractor, SimilarityEngine, and GestureDetector as helper instances.
- MediaItem contains a Features struct as a value member.



## Appendix B — User Manual

---

### Setup

Place image files (.jpg, .jpeg, .png) and video files (.mp4, .mov, .avi) in the folder `bin/data/`. The application automatically detects and loads all supported files on startup. Metadata XML files are generated automatically in `bin/data/metadata/` the first time each file is loaded. To force recomputation of all features, delete the contents of that folder before launching.

### Keyboard controls

| Key                | Action                                                                        |
|--------------------|-------------------------------------------------------------------------------|
| TAB                | Toggle between 2D grid mode and 3D gallery mode                               |
| L                  | Sort / invert frames by mean luminance (dark to light or light to dark)       |
| G                  | Toggle similarity-based grouping; frames are rearranged by group on the walls |
| F                  | Toggle depth fog in 3D mode                                                   |
| M                  | Toggle floating metadata labels on each frame                                 |
| C                  | Toggle live camera overlay with face detection and gesture recognition        |
| SPACE              | Play / pause video (in fullscreen mode)                                       |
| BACKSPACE          | Return from fullscreen to gallery                                             |
| ENTER              | Open the currently selected frame in fullscreen (3D mode)                     |
| Arrow LEFT / RIGHT | Navigate between frames in 3D mode; camera moves in front of each frame       |

### Mouse controls

| Action                | Effect                                          |
|-----------------------|-------------------------------------------------|
| Hover over frame (2D) | Frame enlarges with a yellow border             |
| Click on frame (2D)   | Opens frame in fullscreen                       |
| Hover over frame (3D) | Frame enlarges by 30% with a yellow glow border |
| Drag (3D)             | Rotates the camera                              |
| Scroll (3D)           | Zooms in or out                                 |

### Gesture controls (requires webcam)

Press **C** to enable the camera overlay. Gestures are detected in real time and shown above the camera feed.

| <b>Gesture</b>    | <b>Action</b>                                   |
|-------------------|-------------------------------------------------|
| Swipe left        | Navigate to the next frame                      |
| Swipe right       | Navigate to the previous frame                  |
| Strong motion     | Toggle similarity grouping (same as pressing G) |
| Move forward / up | Open current frame in fullscreen                |

### Visual indicators

Each frame displays coloured bars below it. The **yellow bar** shows mean luminance (0–255). The **blue bar** shows edge density (0–100%). For video items, a **green bar** shows the current real-time motion energy, and an **orange bar** shows the video rhythm index. The frame **border thickness** (8–24 px) encodes luminance variance: high-contrast images receive a thicker border. The frame **rendered size** varies between  $0.75\times$  and  $1.25\times$  the base dimensions according to the image’s visual complexity (keypoints and texture). In fullscreen mode, all feature values are shown numerically alongside the three most similar items in the collection and their distances.